

VIA University College

Kløverly – Projektrapport

Mehmet Sahin (360972) og Chadi El-Khatib (361354)

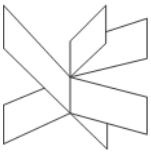
Vejledere: Michael Viuff & Troels Mortensen

[Antal tegn inkl. mellemrum – 61.272]

Softwareingeniør Trainee

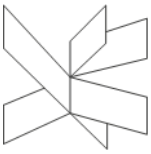
1. semester (SEP1) – reeksamen

16-10-2026

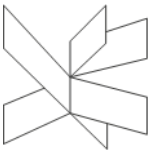


Indhold

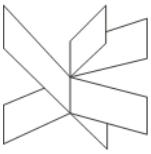
1. Resumé 1
2. Indledning 2
Baggrund og problemfelt 2
Problemformulering 3
Afgrænsning 3
Læsevejledning 4
3. Metode 5
Udviklingsmodel 5
Kravindsamling 6
Sporbarhed 6
Kontrol af sammenhæng 6
Værktøjer 6
4. Analyse 7
Aktører 7
Funktionelle krav 8
Ikke-funktionelle krav 9
Use cases 10
Aktivitetsdiagrammer 15
Domænemodel 16
Persondata og etik 18
5. Design 19
Arkitektur 19
Klassediagram 19



Sekvensdiagrammer 21
Brugerflade 24
Persistens 27
Hjemmeside 27
6. Implementering 29
Opstart og navigation 29
Validering og fejlbeskeder 29
Bytteopgave reserverer point 29
Saldo og nulstilling 30
Redigering med tilbagerulning 31
Afslutning af en opgave 31
Gem og indlæs 31
Data til hjemmesiden 32
7. Test 34
Teststrategi 34
Unittest 34
Black box-test af use cases 36
Test af hjemmesiden 38
Sporbarhedsmatrix 38
Test af ikke-funktionelle krav 39
8. Resultater 41
9. Diskussion 43
Fortolkningen af bytteopgaver 43
Pointlogik i controlleren 43
Data og opgavernes tilstand 43



Hvad mangler i forhold til Bobs ønsker 44
Persondata 44
Test 45
Brug af generativ AI 45
Vandfaldsmodellen på en uge 46
10. Konklusion og anbefalinger 47
Anbefalinger 47
11. Referencer 49
12. Bilag 50



1. Resumé

Denne rapport beskriver udviklingen af et administrationsprogram og en informationshjemmeside til den økologiske landsby Kløverly. Koordinatoren Grønne Bob ønsker at erstatte sin notesbog med et system, der kan registrere grønne tiltag, fællesopgaver og bytteopgaver samt de to typer point, som opgaverne giver: personlige point til den enkelte beboer og fælles point til landsbyens pulje.

Projektet er gennemført efter vandfaldsmodellen med faserne analyse, design, implementering og test. Kravene er udledt af interviewet med Bob og nummereret, så hvert krav kan følges til en use case, til klasser i koden og til en test. Løsningen er et lokalt JavaFX-program skrevet i Java, som gemmer data i en fil på koordinatorens computer. Ud over programmet er der lavet en hjemmeside, hvor JavaScript henter tal og grønne tiltag fra en datafil, som programmet skriver. Hjemmesiden viser ingen navne eller personlige saldi.

Alle 20 funktionelle krav og alle 6 ikke-funktionelle krav blev implementeret og testet med et godkendt resultat. Alle 11 unittests, 15 black box-tests og 4 tests af hjemmesiden bestod. Testene viste dermed, at de implementerede funktioner og pointregler fungerede som forventet.

De vigtigste begrænsninger er, at skærmvisningen til fælleshuset, mål for fællespuljen og et pointboost til sjældne deltagere ikke er implementeret, og at en del af pointlogikken ligger i brugerfladens controller frem for i modellen.

2. Indledning

Baggrund og problemfelt

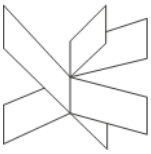
Kløverly er en økologisk landsby med 42 beboere, som dyrker egne grøntsager og deler ting og tjenester med hinanden (VIA University College, 2025a). Koordinatoren Grønne Bob holder i dag styr på landsbyens aktiviteter i en notesbog. Han ønsker et system, der kan synliggøre "det usynlige arbejde": de grønne handlinger og tjenester, som beboerne bidrager med.

I interviewet beskriver Bob tre typer opgaver, som skal behandles forskelligt:

- **Grønne tiltag** er noget, en beboer gør for at leve mere bæredygtigt, fx at cykle på arbejde i en uge. De giver point til en fælles pulje, som Bob kalder "klimakapital", og skal vises uden navne som et inspirationskatalog.
- **Fællesopgaver** er opgaver for fællesskabet, fx madlavning til fællesspisning eller havetjans. De giver personlige point til den beboer, der udfører opgaven.
- **Bytteopgaver** er ydelser eller ting, som beboerne bytter, fx vinduespudsning eller gulerødder. De betales med personlige point.

Bob og eventuelt Don er de eneste, der skal bruge programmet, og det skal kun ligge på Bobs computer. Beboerne kommer til Bob for at få registreret deres opgaver. Derudover ønsker Bob en hjemmeside, hvor alle kan læse om Kløverly og blive inspireret, og en stor skærm i landsbyen, der kan vise opgaverne og point. Bob har også en række idéer, som han ikke har gjort færdige: nulstilling af personlige point med faste intervaller, mål for fællespuljen, som låser op for en fælles aktivitet, og et pointboost på 10–30 % til beboere, der sjældent deltager. Endelig ønsker han, at systemet bruger så lidt energi som muligt.

Problemet er altså, at Kløverly mangler et overblik over, hvem der bidrager med hvad, og et pointsystem, der kan holde styr på to forskellige slags point uden at fællesskabet bliver til en konkurrence.



Problemformulering

Hvordan kan vi udvikle et IT-system, der hjælper koordinatoren i Kløverly med at registrere landsbyens tre typer opgaver og de to typer point, og som samtidig formidler de grønne tiltag til omverdenen uden at udstille den enkelte beboer?

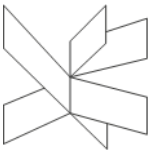
Underspørgsmål:

- Hvilke krav kan udledes af interviewet med Grønne Bob, og hvordan kan de beskrives, så de kan testes?
- Hvordan kan pointreglerne for de tre opgavetyper modelleres, så personlige point aldrig bliver negative, og historikken bevares?
- Hvordan kan data fra programmet vises på en hjemmeside ved hjælp af JavaScript uden at vise personoplysninger?
- Hvordan kan vi dokumentere, at løsningen opfylder kravene?

Afgrænsning

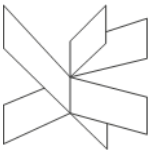
Projektet er gennemført på én uge, og vi har derfor prioriteret de funktioner, som Bob beskriver som nødvendige: registrering af beboere, de tre opgavetyper, de to slags point og hjemmesiden. Følgende er bevidst udeladt:

- **Login og adgang for beboerne.** Bob siger, at kun han og Don skal bruge programmet, og at det kun skal ligge på hans computer.
- **Visning på den store skærm.** Programmet har ingen særskilt skærmvisning.
- **Mål for fællespuljen**, som låser op for en fælles aktivitet, fx en regnvandsopsamler.
- **Pointboost til sjældne deltagere.** Bob har ikke selv besluttet reglen. Koordinatoren kan i stedet give ekstra point manuelt via en pointkorrektion (FR10).
- **Nulstilling med faste intervaller.** Koordinatoren kan nulstille en beboers point manuelt (FR18), men programmet gør det ikke automatisk.
- **Indrapportering fra beboerne selv**, fx via hjemmesiden. Hjemmesiden viser kun information.



Læsevejledning

Rapporten er bygget op efter VIA's guide til projektrapporter (VIA University College, 2024). Kapitel 3 beskriver, hvordan vi har grebet projektet an. Kapitel 4 er analysen med aktører, krav, use cases, aktivitetsdiagrammer og domænemodel. Kapitel 5 handler om designet, og kapitel 6 viser de dele af koden, der er vigtigst for at forstå løsningen. I kapitel 7 dokumenterer vi testene, og kapitel 8 samler resultaterne. Kapitel 9 diskuterer svaghederne ved løsningen, og kapitel 10 besvarer problemformuleringen og giver anbefalinger til det videre arbejde. Kravene og use casene har ID'er (FR01–FR20, NFR01–NFR06 og UC01–UC19), som går igen gennem hele rapporten, så man kan følge et krav fra analysen helt til testen.



3. Metode

Dette kapitel beskriver, hvordan projektet er gennemført. Selve samarbejdet og vores refleksioner over arbejdsformen er beskrevet i procesrapporten.

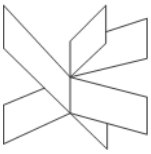
Udviklingsmodel

Projektet følger vandfaldsmodellen, hvor arbejdet gennemføres i faser, der kommer efter hinanden: krav og analyse, design, implementering og test (Royce, 1970). Hver fase bygger på resultatet af den forrige. Vi valgte modellen, fordi den er en del af kursets læringsmål (VIA University College, 2025b), og fordi kravene var forholdsvis faste, når interviewet med Bob først var analyseret. Royce (1970) påpeger selv, at det er risikabelt først at teste til sidst. Vi har derfor tilladt små tilbageløb, fx da vi under designet tilføjede et krav om at afslutte en opgave.

Projektet blev gennemført på én uge i oktober 2026:

Dag	Fase	Resultat
Mandag 12/10	Analyse og design	Brainstorm og aktører, kravspecifikation, domænemodel, klassediagram, sekvensdiagrammer, Maven-projekt og modelklasser
Tirsdag 13/10	Implementering	JavaFX-brugerflade, persistens og hjemmeside
Onsdag 14/10	Analyse (opdatering) og implementering	Use case-diagram og use case-beskrivelser, revideret kravspecifikation, hjemmeside med JavaScript
Torsdag 15/10	Test og dokumentation	Unittests, black box-tests og test af hjemmesiden gennemført. Sporbarhedsmatrix, diagrammer, kravspecifikation, use case-beskrivelser og projektrapport rettet og færdiggjort.
Fredag 16/10	Aflevering	Projektrapport, procesrapport og kildekode afleveret kl. 13.00

Tabel 1: Projektets faser og resultater.



Kravindsamling

Kravene kommer fra interviewet med Grønne Bob i casen. Vi gik interviewet igennem sætning for sætning og markerede hvert sted, hvor Bob siger noget, systemet skal kunne, eller en betingelse, det skal overholde. Hvert ønske blev skrevet om til et funktionelt krav på formen "Systemet skal kunne …", og vi skrev citatet fra interviewet ved siden af (tabel 2). Nogle steder var Bob ikke selv afklaret, fx med pointboostet og målene for fællespuljen. Der har vi hellere afgrænset os eller ladet koordinatoren gøre det manuelt end at finde på regler, som Bob ikke har godkendt. De ikke-funktionelle krav er skrevet, så hvert krav kan måles i en test (tabel 3). Derefter blev kravene omsat til use cases. Et enkelt krav, FR20, kom først til under designet, da vi besluttede, at point først gives, når en opgave er udført.

Sporbarhed

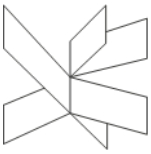
Alle funktionelle krav har et ID (FR01–FR20), og alle use cases har et ID (UC01–UC19). ID'erne går igen i use case-beskrivelserne, i testene og i sporbarhedsmatricen. Sporbarhedsmatricen i kapitel 7 viser for hvert krav, hvilken use case, hvilken klasse og hvilken test der hører til. På den måde kan man følge et krav fra Bobs ønske til den test, der viser, at det er opfyldt.

Kontrol af sammenhæng

Ud over testene har vi kontrolleret, at dokumenterne passer sammen. Sidst i ugen gennemgik vi diagrammerne mod koden ét for ét: klasse- og metodenavne i klassediagrammet, livslinjer og beskeder i sekvensdiagrammerne og navnene på use casene i diagrammet, i beskrivelserne og i rapporten. Hver uoverensstemmelse blev rettet det sted, hvor fejlen var, og som regel var det dokumentationen og ikke koden, der skulle rettes. Et eksempel er UC16, som hed "Se rapport" i use case-diagrammet, men "Se rapport over opgaver og point" i beskrivelserne.

Værktøjer

- **Java 21 og JavaFX 21** til programmet, med FXML til skærmbillederne (Oracle, 2023; OpenJFX, u.å.).



- **Maven** til at styre afhængigheder og bygge projektet, og **JUnit 5** til test af modelklasserne (The JUnit Team, u.å.).
- **Astah** til UML-diagrammer.
- **HTML, CSS og JavaScript** til hjemmesiden.
- **Git og GitHub** til versionsstyring, så begge gruppemedlemmer kunne arbejde i samme kodebase.
- **Generativ AI (ChatGPT og Claude)** som hjælp til kode, diagrammer og tekst (OpenAI, 2026; Anthropic, 2026). Generativ AI blev brugt til sparring om kode, UML-diagrammer, fejlfinding og formulering af rapporttekst. Forslagene blev gennemgået, tilpasset og testet af gruppen.

4. Analyse

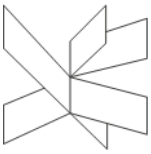
Analysen bygger på interviewet med Grønne Bob (VIA University College, 2025a) og vores brainstorm over aktører, se bilag 1.

Aktører

Ud fra interviewet har vi fundet tre aktører:

- **Koordinator (Bob / Don)** er den eneste aktør, der bruger administrationsprogrammet. Koordinatoren registrerer beboere og opgaver og giver point. Bob siger, at beboerne skal "komme til mig for at få registreret deres opgaver".
- **Beboer** bruger kun hjemmesiden til at læse om Kløverly og finde inspiration i de grønne tiltag.
- **Ekstern besøgende** er fx et andet fællesskab, der vil lade sig inspirere. Den eksterne besøgende bruger også kun hjemmesiden.

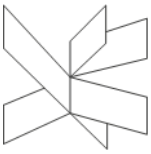
Beboerne er altså aktører på hjemmesiden, men ikke i administrationsprogrammet. I programmet findes beboerne kun som data, som koordinatoren registrerer. Den skelnen er vigtig, fordi den afgør, hvem der må se hvad: Koordinatoren ser navne og point, mens beboere og besøgende kun ser anonyme oplysninger.



Funktionelle krav

Tabel 2 viser de funktionelle krav. For hvert krav angiver vi, hvilket ønske i interviewet kravet bygger på, så det er tydeligt, at kravene kommer fra Bob og ikke fra os. Den fulde kravspecifikation findes i bilag 2.

ID	Krav: Systemet skal kunne …	Grundlag i interviewet
FR01	oprette en beboer med navn, husnummer og fødselsdato.	Programmet skal bruges "med beboere, opgaver, points"
FR02	redigere oplysninger om en eksisterende beboer.	Som FR01
FR03	slette en beboer efter bekræftelse.	Som FR01
FR04	vise en oversigt over registrerede beboere.	Som FR01
FR05	oprette en opgave med type, beskrivelse, dato og point. Ved en bytteopgave vælges en beboer som bestiller.	"registrering af de tre typer af opgaver"
FR06	redigere en eksisterende opgave.	Som FR05
FR07	slette en opgave efter bekræftelse.	Som FR05
FR08	vise en samlet oversigt over aktive opgaver med type, dato, beskrivelse og point.	"udtræk der viser tilgængelige bytteopgaver"
FR09	vise den valgte beboers aktuelle personlige pointsaldo.	"personlige og individuelle points"
FR10	korrigere en beboers personlige point manuelt via en pointpostering, uden at saldoen kan blive negativ.	Nye opgaver "må fastsættes ved et skøn"; idéen om et boost
FR11	vise Kløverlys fælles pointpulje.	"points til en fælles pulje"
FR12	gemme beboere, opgaver og point lokalt og indlæse dem igen ved opstart.	"Programmet skal kun være på min computer"
FR13	vise en informationshjemmeside om Kløverly og grøn inspiration uden private beboeroplysninger.	"en hjemmeside … så alle kan læse om os"
FR14	registrere grønne tiltag og tilføje deres point til fællespuljen.	"grønne points … til en fælles pulje"
FR15	registrere fællesopgaver, som giver en beboer personlige point.	"personlige points for opgaver som hver deltager kan modtage"



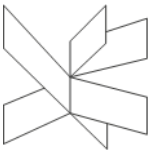
ID	Krav: Systemet skal kunne …	Grundlag i interviewet
FR16	oprette bytteopgaver, hvor bestilleren betaler med personlige point. Når opgaven afsluttes, modtager udføreren pointene.	"de personlige points skal kunne bruges ved bytteopgaver"
FR17	vise grønne tiltag anonymt på hjemmesiden.	"vises uden navne så de fungerer som inspirationskatalog"
FR18	nulstille en beboers personlige point efter bekræftelse og gemme nulstillingen i historikken.	"nulstille de personlige points med faste intervaller"
FR19	vise en samlet rapport over opgaver og point.	"udtræk"
FR20	afslutte en aktiv opgave, så pointene registreres efter opgavetypen.	Afledt: point gives først, når en opgave er udført

Tabel 2: Funktionelle krav.

Ikke-funktionelle krav

De ikke-funktionelle krav er formuleret, så de kan måles i en test.

ID	Krav	Sådan måles kravet
NFR01	Koordinatoren skal kunne nå Beboere, Opgaver, Point og Rapport med ét klik fra menuen.	Tæl klik fra menuen til hvert skærmbillede.
NFR02	Ugyldige eller manglende indtastninger skal give en fejlbesked, der forklarer fejlen, uden at programmet lukker.	Indtast tomme felter, bogstaver i talfelter og 0 point.
NFR03	En ændring skal kunne ses i oversigterne, så snart den er gemt, uden genstart.	Opret en opgave og kontrollér listen med det samme.
NFR04	Data, der er gemt, skal være tilgængelige efter genstart af programmet.	Luk og genstart programmet.
NFR05	Programmet skal kunne bruges uden login og uden internetforbindelse.	Start programmet med netværket slået fra.
NFR06	Hjemmesiden må ikke vise navne, husnumre, individuelle pointsaldi eller andre oplysninger, der identificerer beboerne.	Gennemse hjemmesiden og datafilen data.json.



Tabel 3: Ikke-funktionelle krav.

Use cases

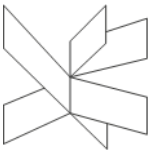
Kravene er omsat til 19 use cases. Figur 1 viser use case-diagrammet. Diagrammet har to systemgrænser, fordi løsningen består af to dele: "Kløverly – Administrationsprogram", som kun koordinatoren bruger, og "Kløverly – Hjemmeside", som beboere og eksterne besøgende bruger. Inden for administrationsprogrammet er use casene samlet i tre pakker: Beboerhåndtering (UC01–UC04), Opgavehåndtering (UC05–UC08, UC12–UC14 og UC19) samt Point og rapport (UC09–UC11, UC15 og UC16). Pakkerne gør diagrammet lettere at læse og viser, hvilke dele af programmet use casene hører til.

Tabel 4 viser, hvilke krav hver use case dækker. FR12 (gem og indlæs) har ingen use case, fordi koordinatoren ikke selv starter den. Den sker automatisk ved hver ændring og ved opstart, og den testes derfor særskilt (test T15).



Figur 1: Use case-diagram.

Use case	Krav	Aktør
UC01 Opret beboer · UC02 Rediger beboer · UC03 Slet beboer · UC04 Se beboere	FR01–FR04	Koordinator
UC05 Opret opgave · UC06 Rediger opgave · UC07 Slet opgave · UC08 Se aktive opgaver	FR05–FR08	Koordinator
UC09 Se personlige point · UC10 Manuel pointkorrektion · UC11 Se fælles pointpulje	FR09–FR11	Koordinator
UC12 Opret grønt tiltag	FR14	Koordinator
UC13 Opret fællesopgave	FR15	Koordinator

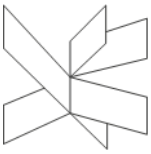


Use case	Krav	Aktør
UC14 Opret bytteopgave	FR16	Koordinator
UC15 Nulstil personlige point	FR18	Koordinator
UC16 Se rapport over opgaver og point	FR19	Koordinator
UC17 Se information om Kløverly	FR13	Besøgende
UC18 Se grønne tiltag anonymt	FR17	Besøgende
UC19 Afslut opgave	FR20 (og FR14–FR16)	Koordinator

Tabel 4: Use cases og de krav, de dækker. Alle beskrivelser findes i bilag 3.

UC19 Afslut opgave er den mest komplekse use case, fordi pointene behandles forskelligt efter opgavetypen. Den er derfor vist i fuld længde i tabel 5. Alternativerne er nummereret efter det trin i hovedscenariet, hvor de opstår.

UC19	Afslut opgave
Krav	FR20, FR14, FR15, FR16
Primær aktør	Koordinator
Forbetingelse	Der findes mindst én aktiv opgave.
Efterbetingelse	Opgaven er ikke længere aktiv. Pointene er registreret efter opgavetypen og gemt.
Hovedscenarie	1. Koordinatoren åbner Opgaver. 2. Koordinatoren vælger en aktiv opgave. 3. Koordinatoren vælger Afslut opgave. 4. Systemet viser en liste over beboere og beder om den beboer, der udførte opgaven. 5. Koordinatoren vælger beboeren. 6. Systemet registrerer en pointpostering med opgavens point hos beboeren. 7. Systemet fjerner opgaven fra de aktive opgaver. 8. Systemet gemmer ændringerne og opdaterer hjemmesidens datafil.

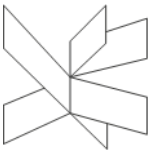


UC19	**Afslut opgave**
Alternativer	2a. Ingen opgave er valgt: Systemet viser "Vælg en opgave først." Use casen slutter. 4a. Opgaven er en bytteopgave: Bestilleren vises ikke på listen, da bestilleren ikke kan betale sig selv. Bestillerens betaling blev trukket, da opgaven blev oprettet (UC14). 4b. Opgaven er en bytteopgave, og der findes ingen anden beboer: Systemet viser en fejlbesked. Use casen slutter. 4c. Opgaven er et grønt tiltag: Systemet beder om bekræftelse i stedet for en beboer. Ved bekræftelse registreres pointene i fællespuljen, og der fortsættes fra trin 7. 4d. Opgaven er en fællesopgave, og der er ingen registrerede beboere: Systemet viser "Der er ingen beboere registreret." Use casen slutter. 5a. Koordinatoren annullerer: Intet ændres. Use casen slutter.

Tabel 5: Use case-beskrivelse for UC19 Afslut opgave.

UC14 Opret bytteopgave og UC10 Manuel pointkorrektion er også vist i fuld længde, fordi de indeholder de to vigtigste pointregler: En bytteopgave skal betales med det samme, og en saldo må aldrig blive negativ.

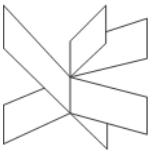
UC14	**Opret bytteopgave**
Krav	FR05, FR16
Primær aktør	Koordinator
Forbetingelse	Bestilleren er registreret som beboer og har personlige point.
Efterbetingelse	Bytteopgaven er gemt som aktiv opgave hos bestilleren, og prisen er trukket fra bestillerens saldo som en negativ pointpostering.



UC14	**Opret bytteopgave**
Hovedscenarie	1. Koordinatoren åbner Opgaver. 2. Koordinatoren vælger "+ Opret opgave". 3. Koordinatoren vælger typen Bytteopgave. 4. Systemet gør det muligt at vælge en beboer. 5. Koordinatoren vælger bestilleren. 6. Koordinatoren indtaster beskrivelse, dato og pris i point og vælger OK. 7. Systemet kontrollerer oplysningerne og beregner bestillerens saldo. 8. Systemet registrerer en negativ pointpostering hos bestilleren. 9. Systemet gemmer bytteopgaven som aktiv opgave og viser den i listen.
Alternativer	5a. Ingen beboer er valgt: Systemet viser en fejlbesked. Der fortsættes fra trin 5. 6a. Koordinatoren annullerer: Intet ændres. Use casen slutter. 7a. Point er ikke et tal eller er 0: Systemet viser en fejlbesked. Ingen opgave oprettes. 7b. Bestilleren har for få point: Systemet viser fx "Hanne har kun 10 point og kan ikke betale 50 point." Ingen opgave oprettes.

Tabel 6: Use case-beskrivelse for UC14 Opret bytteopgave.

UC10	**Manuel pointkorrektion**
Krav	FR10
Primær aktør	Koordinator
Forbetingelse	Der er registreret mindst én beboer.
Efterbetingelse	Korrektionen er gemt som en pointpostering hos beboeren, og saldoen er ikke negativ.
Hovedscenarie	1. Koordinatoren åbner Point. 2. Koordinatoren vælger en beboer. 3. Systemet viser beboerens saldo og pointposteringer. 4. Koordinatoren vælger "+ Manuel pointkorrektion". 5. Koordinatoren indtaster antal point, positivt eller negativt, og en årsag og vælger OK. 6. Systemet kontrollerer, at tallet er gyldigt og ikke 0. 7. Systemet kontrollerer, at saldoen ikke bliver negativ. 8. Systemet registrerer pointposteringen og gemmer. 9. Systemet viser den nye saldo og posteringen i listen.



UC10	Manuel pointkorrektion
Alternativer	2a. Ingen beboer er valgt i trin 4: Systemet viser "Vælg en beboer først." 5a. Koordinatoren annullerer: Intet ændres. 6a. Point er ikke et tal: Systemet viser "Point skal være et tal." 6b. Point er 0: Systemet viser "Pointændringen må ikke være 0." 7a. Saldoen ville blive negativ: Systemet viser en fejlbesked, og intet registreres.

Tabel 7: Use case-beskrivelse for UC10 Manuel pointkorrektion.

Korrektionen bruges også til at give et ekstra pointboost til en beboer, der sjældent deltager, som Bob foreslår. Reglen for boostet er ikke bygget ind i programmet, fordi Bob ikke selv har fastlagt den. Koordinatoren kan i stedet give boostet manuelt og skrive årsagen, så det fremgår af historikken.

Aktivitetsdiagrammer

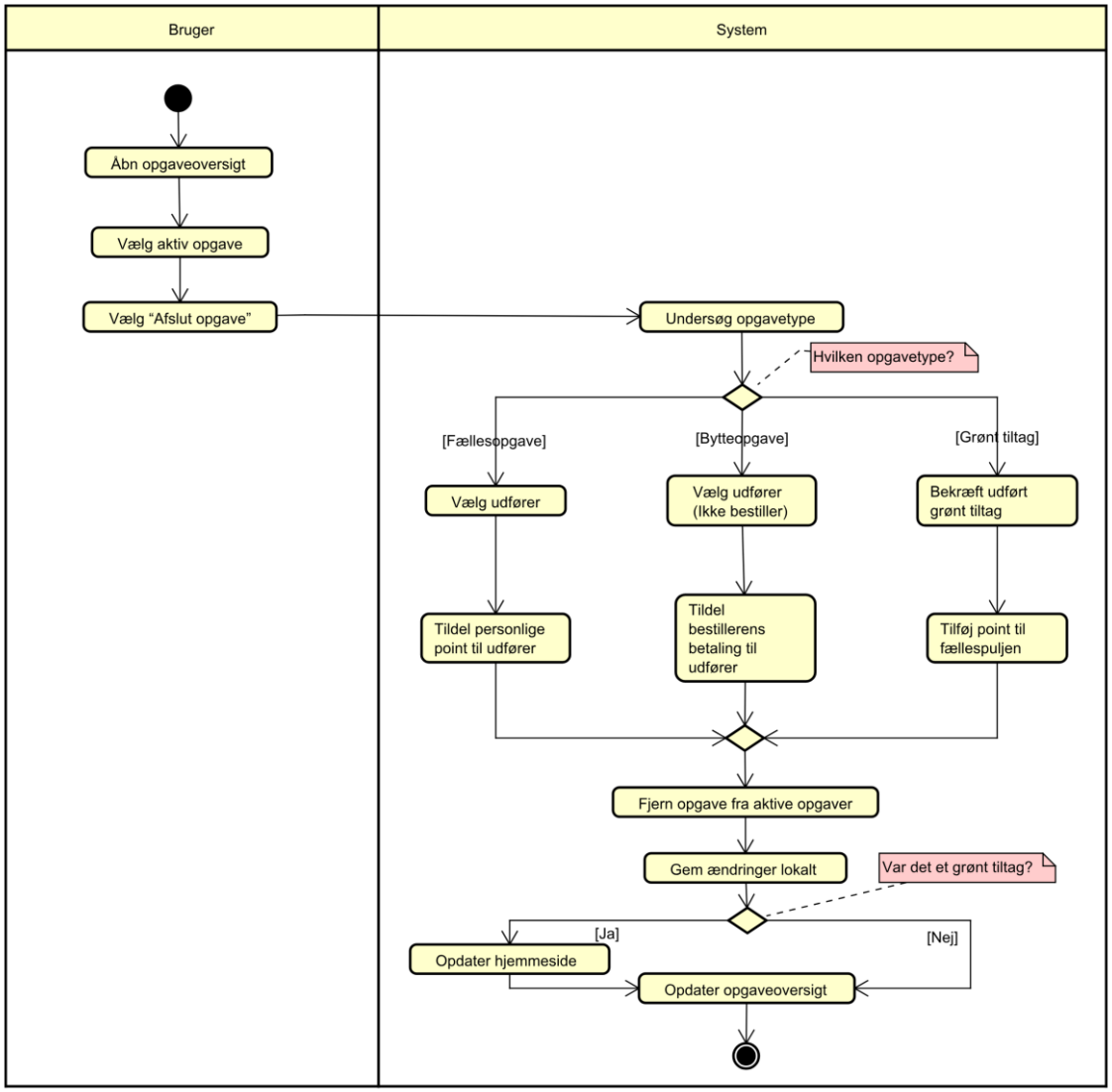
Hver use case har et aktivitetsdiagram, i alt 19 diagrammer, som findes i bilag 4. Diagrammerne er delt i to baner, Bruger og System, så det er tydeligt, hvad koordinatoren gør, og hvad systemet gør. Beslutninger, fx "Gyldige oplysninger?", er vist som beslutningsknuder, og de svarer til alternativerne i use case-beskrivelserne.

Figur 2 viser aktivitetsdiagrammet for UC19 Afslut opgave. Koordinatoren vælger en aktiv opgave og "Afslut opgave", og systemet undersøger opgavetypen. Ved beslutningsknuden "Hvilken opgavetype?" deler forløbet sig i tre grene:

- **Fællesopgave:** Koordinatoren vælger udføreren, og systemet tildeler personlige point til udføreren.
- **Bytteopgave:** Koordinatoren vælger udføreren, som ikke må være bestilleren, og systemet giver udføreren de point, som bestilleren betalte, da opgaven blev oprettet.
- **Grønt tiltag:** Koordinatoren bekræfter, at tiltaget er udført, og systemet tilføjer pointene til fællespuljen.

De tre grene samles i en fletteknude, hvorefter opgaven fjernes fra de aktive opgaver, og ændringerne gemmes. Til sidst spørger diagrammet "Var det et grønt tiltag?", og i så

fald opdateres hjemmesiden. Diagrammet viser dermed, at pointreglerne er forskellige for de tre typer, men at afslutningen ellers foregår ens.

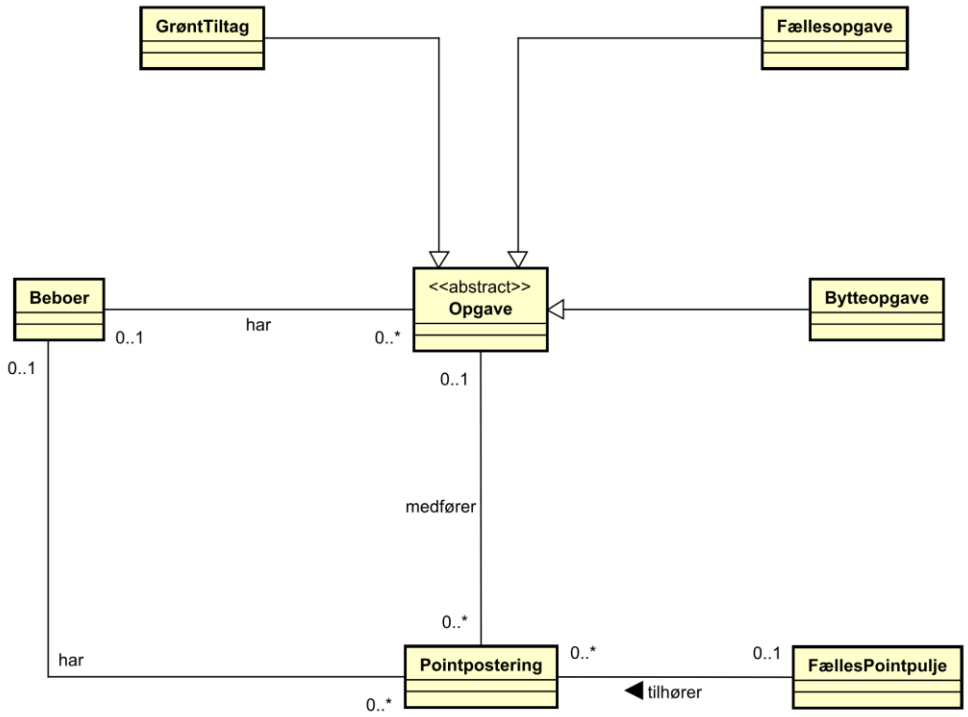


Figur 2: Aktivitetsdiagram for UC19 Afslut opgave.

Domænemodel

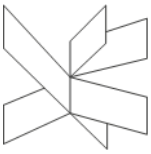
Domænemodellen i figur 3 viser begreberne i Kløverly og deres sammenhæng. Den indeholder kun begreber fra problemdomænet, altså ting som Bob taler om i interviewet, og ingen klasser fra programmet, fx controllere eller filhåndtering. Koordinatoren er ikke

med, fordi koordinatoren er en aktør, der bruger systemet, og ikke noget, systemet gemmer oplysninger om.



Figur 3: Domænemodel.

- **Beboer** har navn, husnummer og fødselsdato. Vi gemmer fødselsdatoen og ikke alderen, fordi alderen ændrer sig og kan beregnes ud fra fødselsdatoen.
- **Opgave** er en generalisering af de tre opgavetyper **Grønt tiltag**, **Fællesopgave** og **Bytteopgave**. Alle har en beskrivelse, en dato og et antal point.
- **Pointpostering** registrerer én ændring af point med dato, antal og årsag. En opgave medfører 0..* pointposteringer, og en beboer har 0..* pointposteringer. Associationerne "har", "medfører" og "tilhører" er navngivet, så diagrammet kan læses som sætninger, fx "en opgave medfører pointposteringer".
- **FællesPointpulje** findes én gang i Kløverly. Pointposteringer fra grønne tiltag tilhører puljen.

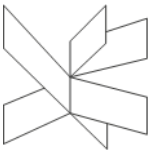


Saldoen gemmes ikke som et tal, men beregnes som summen af posteringerne. Det giver en historik over alle ændringer, som Bob kan bruge, hvis der opstår tvivl, og det gør det muligt at nulstille saldoen uden at slette historikken (FR18).

Persondata og etik

Systemet gemmer navn, husnummer og fødselsdato på beboerne. Det er personoplysninger efter databeskyttelsesforordningen (Europa-Parlamentet og Rådet, 2016). Vi har derfor begrænset, hvor oplysningerne kan ses. De findes kun i administrationsprogrammet på Bobs computer, og hjemmesiden viser kun samlede tal og grønne tiltag uden navne (NFR06).

Bob ønsker et fællesskab "uden konkurrence eller tvang". Et pointsystem kan let komme til at virke som en rangliste. Derfor viser hjemmesiden de grønne tiltag uden navne, og programmet sorterer ikke beboerne efter point. Beboerne bør informeres om, hvilke personoplysninger der registreres, hvorfor de registreres, og hvor længe de opbevares. Hvis en beboer flytter fra Kløverly, bør oplysningerne slettes, når de ikke længere er nødvendige for systemets formål.



5. Design

Arkitektur

Programmet er opdelt i fire dele, så hver del har ét ansvar:

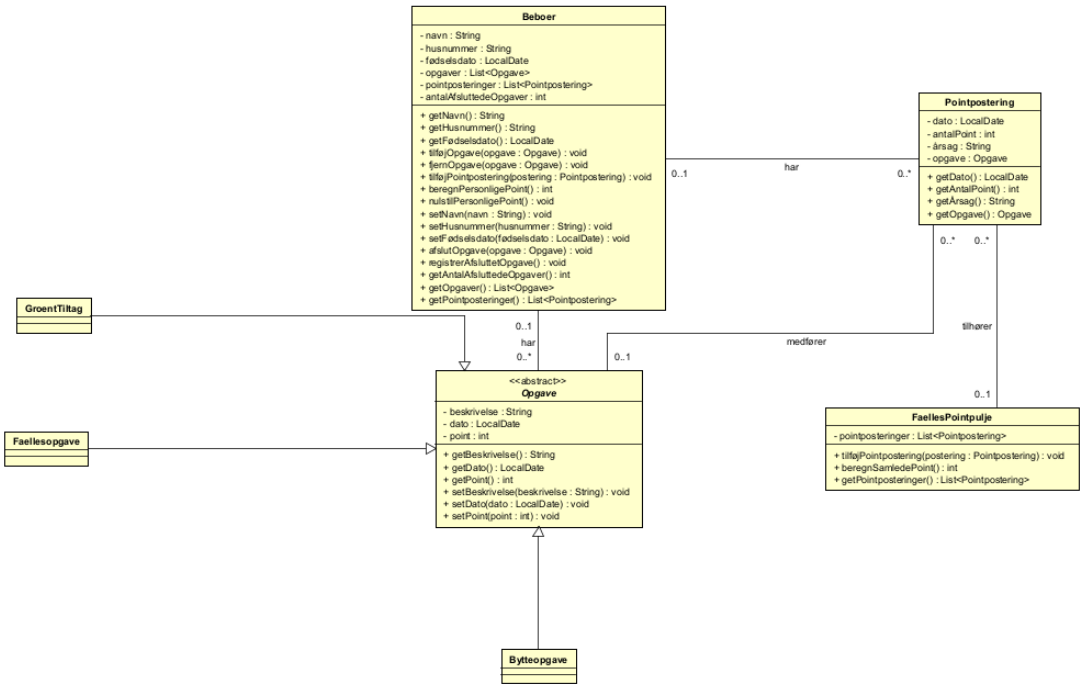
- `kloeverly.model` indeholder domæneklasserne Beboer, Opgave med de tre underklasser, Pointpostering og FaellesPointpulje. Klasserne kontrollerer selv deres regler, fx at et navn er udfyldt, og at en saldo ikke bliver negativ.
- `kloeverly.data` indeholder AppData, der holder programmets data, mens programmet kører, FileDataManager, der gemmer og indlæser filen, og WebDataManager, der skriver datafilen til hjemmesiden. DataContainer samler alle data i ét objekt, så de kan gemmes på én gang.
- `kloeverly.controller` indeholder én controller pr. skærmbillede: MainController (Dashboard), BeboerController, OpgaveController, PointController og RapportController samt SidebarController til menuen.
- **Skærmbillederne** er beskrevet i FXML-filer, og udseendet er samlet i én CSS-fil.

Opdelingen følger idéen i Model-View-Controller: Skærmbilledet (view) viser data, controlleren reagerer på brugerens handlinger, og modellen indeholder data og regler. Fordelen er, at modellen ikke kender brugerfladen. Modelklasserne kan derfor testes med JUnit uden at starte JavaFX, og de samme klasser kan genbruges, hvis der senere laves en skærmvisning til fælleshuset.

Afhængighederne går kun én vej: Controllerne bruger AppData og modelklasserne, AppData bruger FileDataManager og WebDataManager, og modellen bruger ingen af de andre pakker.

Klassediagram

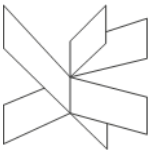
Figur 4 viser klassediagrammet for modellen. Klassediagrammet er udviklet fra domænemodellen: Begreberne er blevet til klasser, attributterne har fået datatyper, og der er tilføjet metoder. Navnene er skrevet uden æ, ø og å, fx FaellesPointpulje, fordi de bruges i Java-koden.



Figur 4: Klassediagram for modellen.

Klasserne har følgende ansvar:

- **Beboer** har navn, husnummer og fødselsdato samt en liste af opgaver og en liste af pointposteringer. Setterne afviser tomme værdier og en fødselsdato i fremtiden. Metoden beregnPersonligePoint() lægger posteringerne sammen, og tilføjPointpostering() afviser en postering, der vil gøre saldoen negativ. tilføjOpgave() trækker prisen for en bytteopgave, og fjernOpgave() giver pointene tilbage, hvis bytteopgaven slettes. nulstilPersonligePoint() registrerer en modpostering, så saldoen bliver 0.
- **Opgave** er en abstrakt klasse med beskrivelse, dato og point. Setterne afviser en tom beskrivelse og point, der ikke er større end 0. Underklasserne **GroentTiltag**, **Faellesopgave** og **Bytteopgave** tilføjer ingen nye attributter. De bruges til at skelne mellem typerne, så programmet kan anvende den rigtige pointregel.



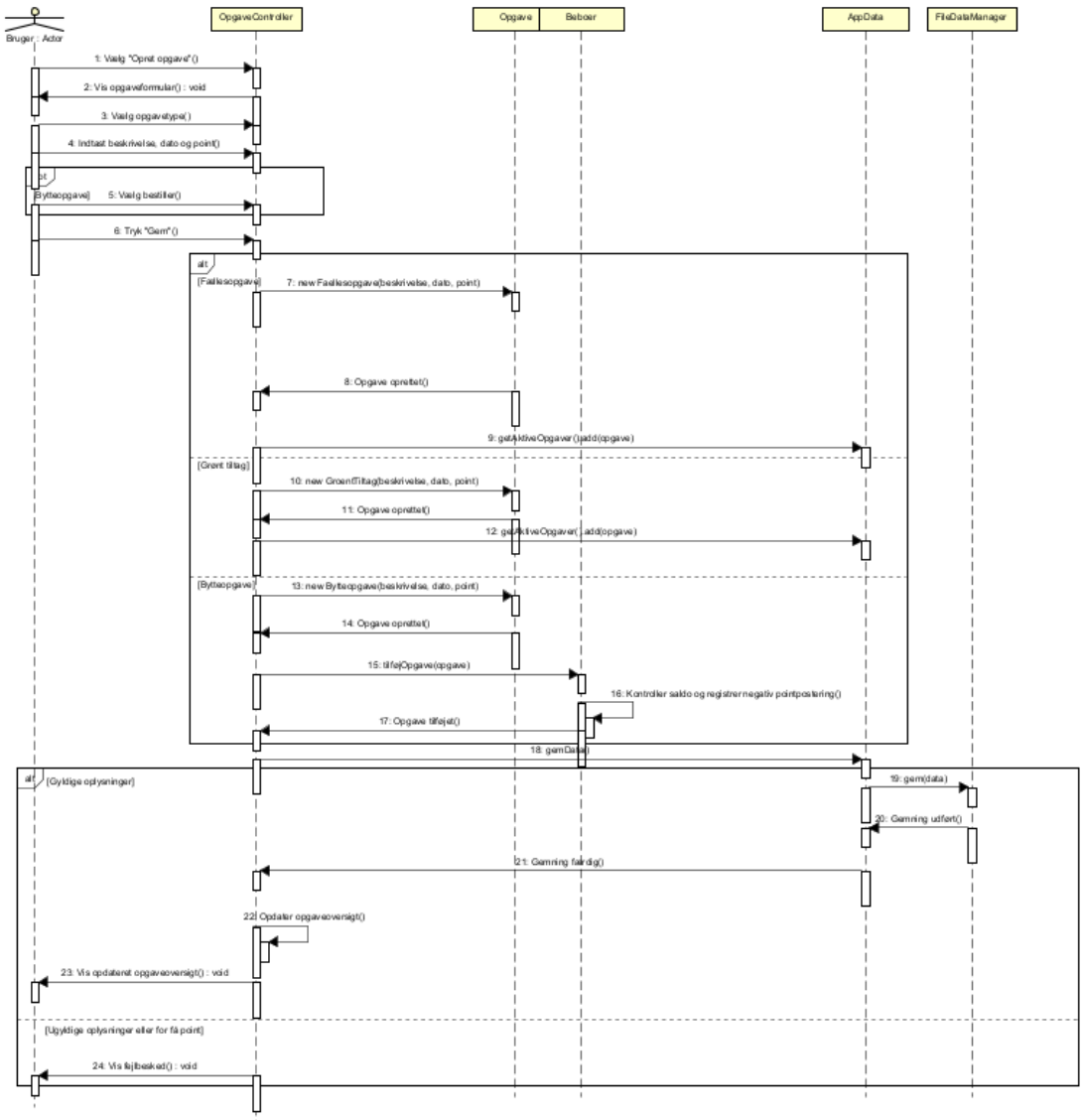
- **Pointpostering** har dato, antal point, årsag og en reference til den opgave, der gav posteringen. Referencen er tom ved manuelle korrektioner og nulstillinger. En postering kan ikke ændres, efter den er oprettet, fordi den er en del af historikken.
- **FaellesPointpulje** har en liste af pointposteringer og metoden beregnSamledePoint(). Der findes kun én pulje, som AppData holder.

Vi overvejede tre alternativer under designet:

- **Saldo som et tal i stedet for posteringer.** En int-attribut "saldo" i Beboer ville være enklere. Men så ville historikken forsvinde, og en nulstilling (FR18) ville slette alle spor af, hvad beboeren har bidraget med. Vi valgte posteringer, selvom saldoen så skal beregnes hver gang. Med 42 beboere og få posteringer pr. beboer er beregningen hurtig.
- **En enum til opgavetypen i stedet for nedarvning.** En attribut "type" ville give én Opgave-klasse. Vi valgte nedarvning, fordi typerne er forskellige begreber i Bobs beskrivelse, og fordi det giver mulighed for at give hver type sin egen opførsel senere.
- **En status på opgaven** (aktiv/afsluttet) i stedet for at fjerne afsluttede opgaver fra listen. Vi valgte at fjerne dem, fordi det var enklest. Ulempen diskuteres i kapitel 9.

Sekvensdiagrammer

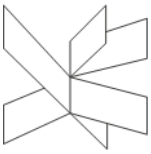
Der er lavet sekvensdiagrammer for UC01 Opret beboer, UC05 Opret opgave og UC19 Afslut opgave. Livslinjerne er de klasser, der findes i koden: controllerne, modelklasserne, AppData, FileDataManager og WebDataManager. Alle kald har aktiveringsbjælker, og svarene er vist med stiplede pile. Figur 5 viser sekvensdiagrammet for UC05, som også dækker bytteopgaver (UC14), fordi alle tre opgavetyper oprettes fra samme dialog. Sekvensdiagrammet for UC01 findes i bilag 4.



Figur 5: Sekvensdiagram for UC05 Opret opgave.

Forløbet i figur 5 er:

- Brugeren vælger "Opret opgave", og OpgaveController viser formularen. Brugeren vælger opgavetype og indtaster beskrivelse, dato og point (besked 1–4). Et opt-fragment viser, at brugeren kun vælger bestiller, når typen er Bytteopgave (5).
- Når brugeren trykker "Gem" (6), deler et alt-fragment forløbet efter opgavetypen. En fællesopgave og et grønt tiltag oprettes og lægges i AppData med



getAktiveOpgaver().add() (7–12). En bytteopgave oprettes og gives til bestilleren med tilføjOpgave(), hvor Beboer selv kontrollerer saldoen og registrerer den negative pointpostering (13–17).

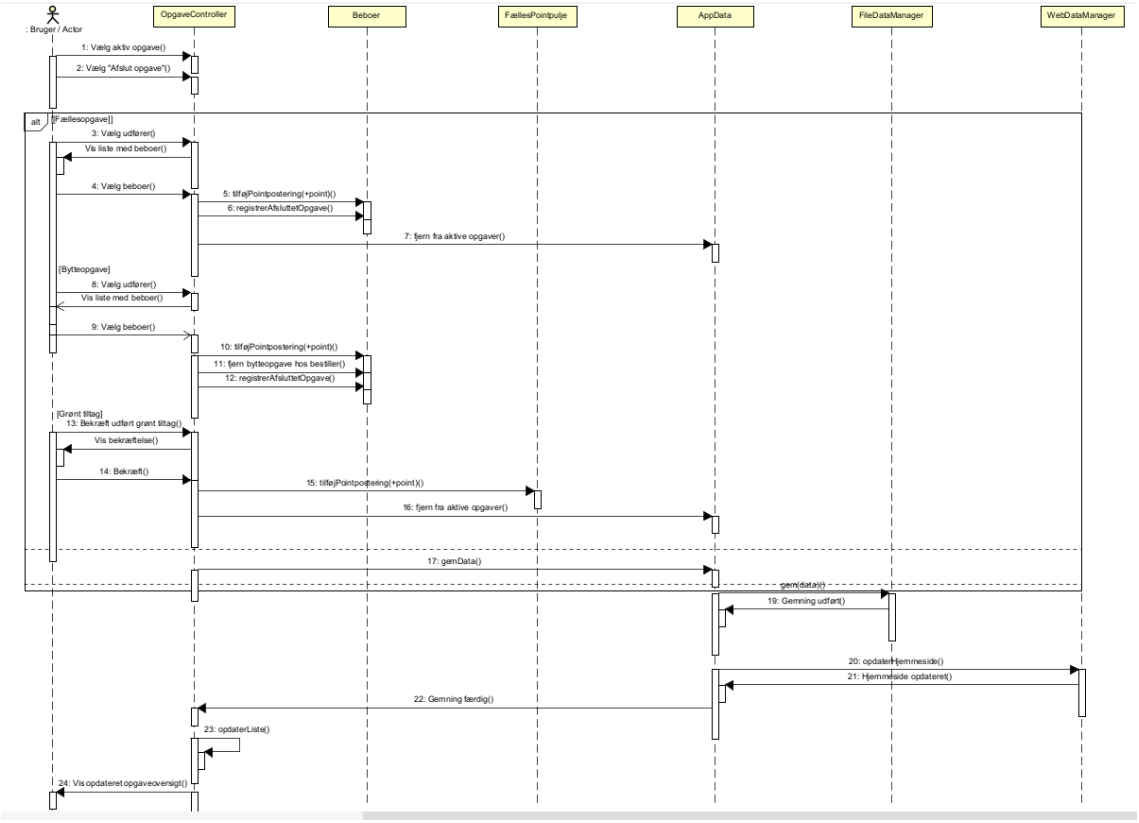
- Derefter kalder controlleren AppData.gemData(), som gemmer via FileDataManager (18–21), og controlleren opdaterer oversigten (22–23). Hvis oplysningerne er ugyldige, eller bestilleren har for få point, vises en fejlbesked (24).

Diagrammet viser, at kontrollen af saldoen ligger i modelklassen Beboer og ikke i controlleren. Det svarer til alternativ 7b i use case-beskrivelsen for UC14 i tabel 6 og er det, der sikrer, at en saldo aldrig bliver negativ.

Figur 6 viser sekvensdiagrammet for UC19 Afslut opgave. Diagrammet har to livslinjer af typen Beboer: udføreren og bestilleren. Ved en bytteopgave er det to forskellige objekter, og pointene skal behandles forskelligt hos dem. Brugeren vælger en aktiv opgave og "Afslut opgave" (besked 1–2). Derefter deler et alt-fragment forløbet i tre:

- **[Fællesopgave]:** Controlleren viser en liste med beboere, og brugeren vælger udføreren. Controlleren kalder tilføjPointpostering() og registrerAfsluttetOpgave() på udføreren og fjerner opgaven fra de aktive opgaver i AppData (3–7).
- **[Bytteopgave]:** Controlleren viser listen uden bestilleren, og brugeren vælger udføreren. Udføreren får pointene med tilføjPointpostering(). Bestilleren får kaldet afslutOpgave(), som fjerner opgaven, men beholder betalingen, og udføreren får kaldet registrerAfsluttetOpgave() (8–12).
- **[Grønt tiltag]:** Brugeren bekræfter, at tiltaget er udført. Controlleren kalder tilføjPointpostering() på FaellesPointpulje og fjerner opgaven fra de aktive opgaver (13–16).

Gemningen ligger efter alt-fragmentet, fordi den sker for alle tre typer. Controlleren kalder AppData.gemData(), som gemmer via FileDataManager og opdaterer hjemmesidens datafil via WebDataManager (17–22). Til sidst opdateres listen (23–24). Diagrammet viser altså, at hjemmesidens datafil skrives, hver gang der gemmes, uanset opgavetype. Aktivitetsdiagrammet i figur 2 viser kun opdateringen af hjemmesiden for grønne tiltag, fordi det kun er de grønne tiltag, der ændrer hjemmesidens liste.

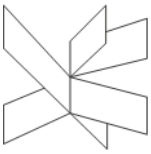


Figur 6: Sekvensdiagram for UC19 Afslut opgave.

Brugerflade

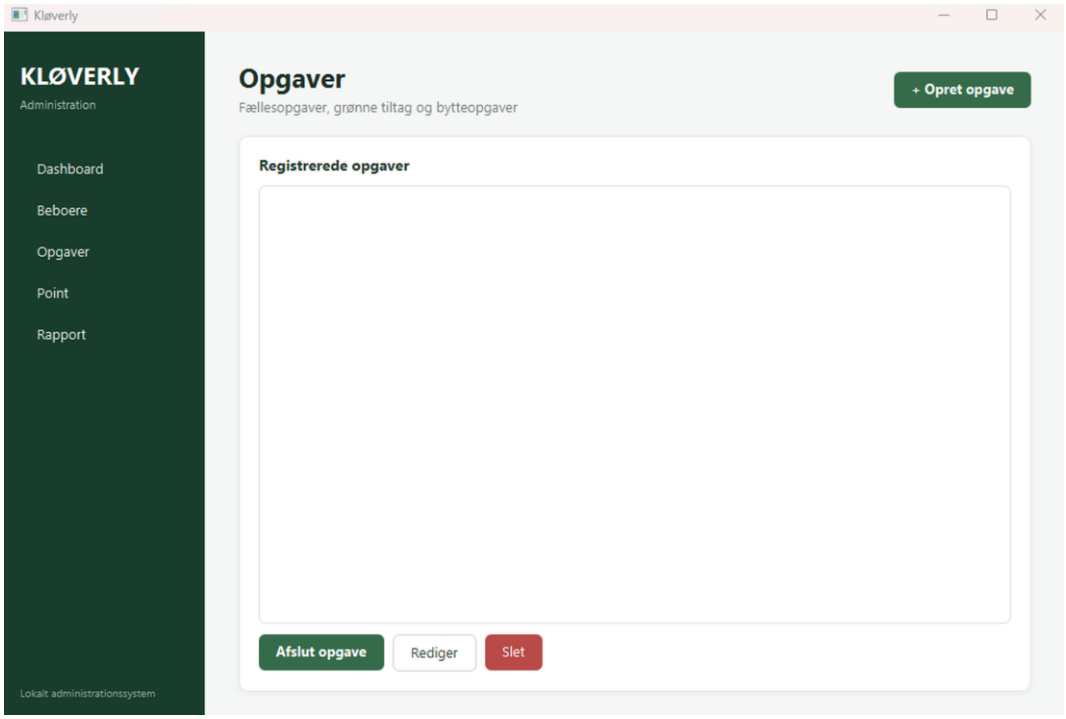
Brugerfladen har en fast menu i venstre side med punkterne Dashboard, Beboere, Opgaver, Point og Rapport. Menupunkterne følger pakkerne i use case-diagrammet: Beboere svarer til Beboerhåndtering, Opgaver til Opgavehåndtering, og Point og Rapport deler pakken Point og rapport. Koordinatoren kan derfor nå alle funktioner med ét klik (NFR01). Alle skærmbilleder er bygget op på samme måde: en overskrift med en kort forklaring, en liste og knapper til handlingerne. Koordinatoren vælger først et element i listen og derefter en handling.

- **Dashboard** giver et overblik med fire tal: antal beboere, antal opgaver, personlige point i alt og fælles point. Under tallene vises de fem nyeste aktive opgaver. Det er det første, Bob ser, når programmet starter.



- **Beboere** viser de registrerede beboere med knapperne "+ Opret beboer", "Rediger" og "Slet".
- **Opgaver** viser alle aktive opgaver af de tre typer med knapperne "+ Opret opgave", "Afslut opgave", "Rediger" og "Slet". Når en opgave oprettes, vælger koordinatoren typen først. Valget af beboer er kun aktivt for bytteopgaver, så koordinatoren ikke kan knytte en fællesopgave til en bestiller ved en fejl.
- **Point** har en liste til valg af beboer, viser beboerens personlige point, fællespuljen og beboerens pointposteringer. Knapperne er "+ Manuel pointkorrektion" og "Nulstil personlige point".
- **Rapport** viser antal beboere, udførte opgaver i alt, nuværende personlige point, fælles point og udførte opgaver pr. beboer (FR19).

Vi har brugt dialogvinduer til at indtaste og bekræfte, i stedet for at have formularer på selve skærmbilledet. Det holder skærmbillederne enkle, og handlinger, der ikke kan fortrydes, fx sletning og nulstilling, kræver altid en bekræftelse (FR03, FR07, FR18). Fejl vises i en dialog med en forklaring, fx "Navn skal udfyldes." (NFR02). Alle skærmbilleder er vist trin for trin i brugermanualen i bilag 6.



Figur 7: Skærmbilledet Opgaver.

Ved designet af brugerfladen har vi brugt fire af Nielsens (1994) principper for brugervenlighed:

- **Konsistens:** Som nævnt er alle skærmbilleder bygget op på samme måde med overskrift, liste og knapper, og knapperne står samme sted. Koordinatoren skal derfor kun lære mønstret én gang.
- **Forebyggelse af fejl:** Valget af beboer er slået fra, når opgaven ikke er en bytteopgave, og bestilleren står ikke på listen over mulige udførere, når en bytteopgave afsluttes. Den slags fejl kan altså slet ikke laves.
- **Synlig status:** Dashboard og Point viser de aktuelle tal, og listerne bliver opdateret, så snart en ændring er gemt (NFR03).
- **Hjælp til at rette fejl:** Fejlbeskederne fortæller, hvad der er galt, fx "Point skal være et tal.", i stedet for at vise en teknisk fejl (NFR02).

Persistens

Alle data samles i et DataContainer-objekt med beboerne, de aktive opgaver og fællespuljen. Objektet gemmes i filen kloeverly-data.bin med Javas objektserialisering, og det sker efter hver ændring. Når programmet starter, indlæses filen, hvis den findes (FR12, NFR04). Da beboerne har deres egne opgaver og pointposteringer, gemmes de automatisk sammen med beboeren.

Vi valgte serialisering, fordi den kræver lidt kode og gemmer hele objektstrukturen, inklusive at en pointpostering peger på den opgave, den kom fra. Alternativet var at gemme i JSON eller XML. Det ville give en fil, som mennesker kunne læse, men det ville kræve, at vi selv skulle omsætte objekterne til tekst og tilbage igen. En database var unødvendig, fordi der kun er én bruger og få data.

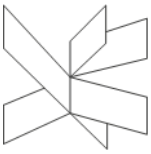
Hjemmeside

Hjemmesiden består af to sider: en forside med information om Kløverly og en side med grønne tiltag. Siderne er statiske HTML-filer med fælles CSS. Tallene og de grønne tiltag hentes med JavaScript fra filen data.json, som programmet skriver, hver gang data gemmes. Figur 8 viser sammenhængen.

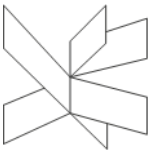


Figur 8: Dataflow fra programmet til hjemmesiden.

Fordi binærfilen ikke kan læses af hjemmesiden, skriver programmet en separat datafil til hjemmesiden. Den indeholder kun de oplysninger, som hjemmesiden må vise: antal beboere, antal udførte opgaver, samlede personlige point, fællespuljen og de grønne tiltag med beskrivelse, dato og point. Navne og husnumre bliver aldrig skrevet til filen, så NFR06 er sikret i datalaget og ikke kun i HTML-koden. Selv hvis nogen åbner data.json direkte, kan de ikke se, hvem der har gjort hvad.



Designet tager også hensyn til Bobs ønske om lavt energiforbrug: Siden bruger ingen frameworks, datafilen er lille, og den hentes kun én gang, når siden åbnes. Siden spørger ikke serveren igen og igen efter nye data.



6. Implementering

Dette kapitel viser udvalgte dele af koden, som er vigtige for at forstå pointreglerne, fejlhåndteringen og hjemmesiden. Hele kildekoden findes på GitHub:

https://github.com/chadiElKhatib/Sep1_Mehmet_Chadi_Reeksamen.git

Opstart og navigation

Programmet starter i KloeverlyApp. Først indlæses data med AppData.hentData(), derefter skrives hjemmesidens datafil, så hjemmesiden altid passer med de gemte data, og til sidst vises Dashboard. Når koordinatoren klikker i menuen, kalder SidebarController metoden KloeverlyApp.skiftScene() med navnet på FXML-filen, fx "OpgaveView.fxml". Metoden indlæser filen og skifter skærmbillede. Hver controller henter selv sine data fra AppData i initialize(), så et skærmbillede altid viser de nyeste data, når det åbnes (NFR03).

Validering og fejlbeskeder

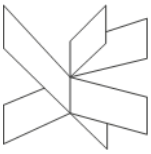
Reglerne for gyldige data ligger i modelklasserne. Hvis en regel brydes, kaster klassen en IllegalArgumentException med en forklaring:

```
public void setNavn(String navn)
{
  if (navn == null || navn.isBlank())
    throw new IllegalArgumentException("Navn skal udfyldes.");
  this.navn = navn.trim();
}
```

Controllerne fanger undtagelsen og viser beskeden i en dialog med metoden visFejl(). Hvis koordinatoren skriver bogstaver i et talfelt, kaster Integer.parseInt() en NumberFormatException, som controlleren omsætter til en forståelig besked, fx "Point skal være et tal.". På den måde lukker programmet aldrig ved en forkert indtastning (NFR02), og den samme regel gælder, uanset hvilket skærmbillede data kommer fra.

Bytteopgave reserverer point

Når en bytteopgave tilføjes til en beboer, kontrollerer Beboer, at saldoen er stor nok, og registrerer en negativ postering:



```
public void tilføjOpgave(Opgave opgave)
{
  if (opgave instanceof Bytteopgave)
  {
    int saldo = beregnPersonligePoint();
    if (opgave.getPoint() > saldo)
      throw new IllegalArgumentException(navn + " har kun " + saldo + " point ...");
    pointposteringer.add(new Pointpostering(opgave.getDato(),
        -opgave.getPoint(), "Bytteopgave: " + opgave.getBeskrivelse(), opgave));
  }
  opgaver.add(opgave);
}
```

Reglen ligger i modellen, så den gælder, uanset hvor i brugerfladen en bytteopgave oprettes. Hvis bytteopgaven slettes, finder fjernOpgave() de posteringer, der hører til opgaven, og fjerner dem, så bestilleren får pointene tilbage.

Saldo og nulstilling

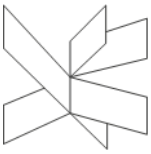
Saldoen gemmes ikke som et tal. Den beregnes hver gang ud fra beboerens posteringer:

```
public int beregnPersonligePoint()
{
  int sum = 0;
  for (Pointpostering postering : pointposteringer)
  {
    sum += postering.getAntalPoint();
  }
  return sum;
}
```

Saldoen og historikken passer derfor altid sammen, fordi point kun bliver registreret ét sted. Nulstillingen (FR18) bygger på samme idé. Den sletter ikke nogen posteringer, men tilføjer en ny postering med minus den nuværende saldo:

```
public void nulstilPersonligePoint()
{
  int saldo = beregnPersonligePoint();
  if (saldo > 0)
  {
    pointposteringer.add(new Pointpostering(LocalDate.now(),
        -saldo, "Nulstilling af personlige point", null));
  }
}
```

Bagefter er saldoen 0, men man kan stadig se i historikken, hvor mange point beboeren havde, og hvornår der blev nulstillet. Har beboeren allerede 0 point, sker der ikke noget, så historikken ikke fyldes op med tomme posteringer. Referencen til en opgave er null, fordi nulstillingen ikke kommer fra en opgave.



Redigering med tilbagerulning

Når en opgave redigeres, kan typen, bestilleren og prisen ændre sig. OpgaveController løser det ved at fjerne den gamle version og tilføje en ny. Hvis den nye version ikke kan tilføjes, fx fordi den nye bestiller har for få point, lægges den gamle version tilbage:

```
try
{
  if (nyOpgave instanceof Bytteopgave)
    nyEjer.tilføjOpgave(nyOpgave);
  else
    AppData.getAktiveOpgaver().add(nyOpgave);
}
catch (IllegalArgumentException e)
{
  if (gammelErAktiv) AppData.getAktiveOpgaver().add(valgt);
  else if (gammelEjer != null) gammelEjer.tilføjOpgave(valgt);
  throw e;
}
```

Det sikrer, at en mislykket redigering ikke efterlader data i en halv tilstand, hvor den gamle opgave er slettet, men den nye ikke er oprettet.

Afslutning af en opgave

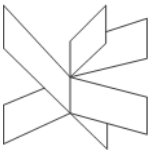
Metoden afslutOpgave() i OpgaveController undersøger opgavetypen med instanceof og registrerer pointene forskelligt, som vist i aktivitetsdiagrammet i figur 2. For en bytteopgave fjernes bestilleren fra listen over mulige udførere, før dialogen vises:

```
List<Beboer> muligeUdfoerere = new ArrayList<>(AppData.getBeboere());
muligeUdfoerere.remove(bestiller);   // bestilleren kan ikke betale sig selv
...
udfoerer.tilføjPointpostering(new Pointpostering(valgt.getDato(),
    valgt.getPoint(), "Udført bytteopgave: " + valgt.getBeskrivelse(), valgt));
bestiller.afslutOpgave(valgt);
udfoerer.registrerAfsluttetOpgave();
AppData.gemData();
```

bestiller.afslutOpgave() fjerner opgaven fra bestillerens liste, men beholder bestillerens negative postering, fordi betalingen nu er gennemført. registrerAfsluttetOpgave() tæller antallet af udførte opgaver, som bruges i rapporten og på hjemmesiden. Her ligger en del af pointreglerne i controlleren og ikke i modellen. Det diskuteres i kapitel 9.

Gem og indlæs

FileDataManager gemmer hele DataContainer-objektet med Javas objektserialisering:



```
public static void gem(DataContainer data)
{
  try (ObjectOutputStream output =
      new ObjectOutputStream(new FileOutputStream(FILNAVN)))
  {
    output.writeObject(data);
  }
  catch (IOException e)
  {
    e.printStackTrace();
  }
}
```

Try-with-resources sørger for, at filen bliver lukket, også hvis der sker en fejl undervejs. Alle klasser, der gemmes, implementerer Serializable. JavaFX' ObservableList kan ikke serialiseres, så DataContainer kopierer listerne over i almindelige ArrayList-objekter, før de gemmes. Når programmet starter, læser hent() filen igen. Findes filen ikke, fx første gang programmet startes, returnerer hent() null, og programmet starter med tomme lister. Hvis gemningen fejler, bliver fejlen kun skrevet i konsollen. Det vender vi tilbage til i kapitel 9.

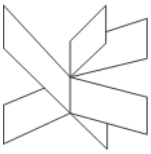
Data til hjemmesiden

AppData.gemData() kalder både FileDataManager.gem() og WebDataManager.opdaterHjemmeside(). WebDataManager beregner de samlede tal og finder de grønne tiltag i fællespuljen, sorterer dem med de nyeste først, fjerner dubletter med samme beskrivelse og skriver højst 20 til data.json. Tekst fra brugeren bliver escaped, så fx anførselstegn i en beskrivelse ikke ødelægger filen.

Filen bygges som tekst med en StringBuilder. Hvert grønt tiltag skrives sådan:

```
json.append("    { \"beskrivelse\": \"")
    .append(escapeJson(tiltag.getBeskrivelse()))
    .append("\", \"dato\": \"")
    .append(tiltag.getDato())
    .append("\", \"point\": ")
    .append(tiltag.getPoint())
    .append(" }");
```

Der er kun felter til beskrivelse, dato og point. Der findes ikke noget felt til et navn, og derfor kan hjemmesiden heller ikke komme til at vise, hvem der har lavet tiltaget.



På hjemmesiden henter kloeverly.js filen med fetch (Mozilla, u.å.) og indsætter værdierne i siden:

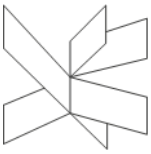
```
fetch("data.json")
  .then(function (svar) { return svar.json(); })
  .then(function (data)
  {
    visTal(data);
    visGroenneTiltag(data.groenneTiltag);
  })
  .catch(function (fejl) { visFejl(); });
```

Begge sider bruger samme JavaScript-fil, og hver funktion udfylder kun de elementer, der findes på den aktuelle side. De grønne tiltag indsættes med textContent og ikke med innerHTML, så en beskrivelse altid vises som tekst og ikke kan indeholde HTML-kode. Browsere tillader ikke fetch fra en fil, der er åbnet direkte fra disken. Siden skal derfor åbnes gennem en lokal webserver, fx den indbyggede server i IntelliJ. Hvis data ikke kan hentes, viser siden en besked om det i stedet for tomme felter.

Hvert grønt tiltag bliver vist som et lille kort. Kortet bygges med DOM-metoder i stedet for at sætte HTML-tekst sammen (forkortet):

```
tiltag.forEach(function (t)
{
  var kort = document.createElement("article");
  kort.className = "tiltag-card";
  var titel = document.createElement("h3");
  titel.textContent = t.beskrivelse;
  var info = document.createElement("div");
  info.appendChild(lavLinje("Dato", formaterDato(t.dato), ""));
  info.appendChild(lavLinje("Bidrag til fællesskabet",
      "+" + t.point + " point", "point-value"));
  kort.appendChild(titel);
  kort.appendChild(info);
  liste.appendChild(kort);
});
```

formaterDato() laver datoen om fra formatet i data.json, fx "2026-10-13", til "13. oktober 2026", så den er nem at læse for de besøgende. Hvis der endnu ikke er nogen grønne tiltag, viser siden en kort besked i stedet for et tomt område, så man kan se, at siden virker.



7. Test

Løsningen er testet med unittests af modelklasserne, black box-test af use cases gennem brugerfladen, test af hjemmesiden og test af de ikke-funktionelle krav.

Teststrategi

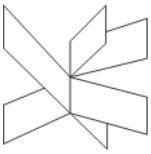
Vi har testet på to niveauer. Unittestene kontrollerer modelklasserne direkte. Det er der, pointreglerne ligger, og de kan testes uden at starte brugerfladen. Black box-testene kontrollerer hele programmet gennem brugerfladen ud fra use case-beskrivelserne. Her ser testeren kun på input og forventet resultat og ikke på koden.

Testdataene er valgt, så vi prøver både gyldige og ugyldige input, og så testene rammer pointreglerne, hvor fejl er mest sandsynlige. Hanne har fx 20 point, da hun bestiller en bytteopgave til 10 point (T04). Det er inden for saldoen, så den skal accepteres. Bagefter har hun 10 point tilbage, og en bytteopgave til 50 point (T05) skal derfor afvises. På samme måde prøver T09 en korrektion, der ville gøre saldoen negativ. Det forventede resultat er taget fra use casens efterbetingelse eller fra den alternative vej, som testen udløser.

Unittest

Modelklasserne er testet med JUnit 5 i klassen BeboerTest. Testene dækker pointreglerne, som er den mest kritiske del af programmet. Pointene for en afsluttet opgave registreres i testene med tilføjPointpostering(), ligesom OpgaveController gør, når en opgave afsluttes.

Test	Krav	Kontrollerer
nyBeboerHarNulPoint	FR09	En ny beboer har saldo 0.
faellesopgaveGiverPointVedAfslutning	FR15, FR20	En afsluttet fællesopgave giver point, og antallet af udførte opgaver stiger.
bytteopgaveTraekkerPointVedOprettelse	FR16	En bytteopgave trækker prisen fra bestilleren, når den oprettes.
bytteopgaveAfvisesVedForFaaPoint	FR16	En bytteopgave afvises, hvis saldoen er for lille.



Test	Krav	Kontrollerer
sletningAfBytteopgaveTilbagefoererPoint	FR07	Sletning af en bytteopgave giver bestilleren pointene tilbage.
afsluttetBytteopgaveBeholderBetalingen	FR16, FR20	Bestilleren beholder betalingen, når bytteopgaven afsluttes, og pointene trækkes ikke igen
nulstillingGiverNulOgBevarerHistorik	FR18	Nulstilling giver saldo 0 og bevarer posteringerne.
negativPointposteringAfvisesHvisSaldoBliverNegativ	FR10	En korrektion, der gør saldoen negativ, afvises.
registrerAfsluttetOpgaveOegerAntal	FR19	Antallet af udførte opgaver tælles op.
opgaveMedNulPointAfvises	FR05	En opgave med 0 point afvises.
beboerUdenNavnAfvises	FR01, NFR02	En beboer uden navn afvises.

Tabel 8: Unittests i BeboerTest.

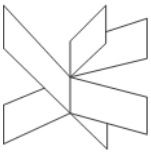
Testene følger samme mønster: Først får beboeren et kendt antal point, så udføres handlingen, og til sidst kontrolleres resultatet. Her er testen af, at en bytteopgave afvises, når bestilleren har for få point:

```
@Test
void bytteopgaveAfvisesVedForFaaPoint()
{
  hanne.tilføjPointpostering(
      new Pointpostering(idag, 10, "Manuel pointkorrektion", null));
  Bytteopgave bytteopgave = new Bytteopgave("Vinduespudsning", idag, 15);

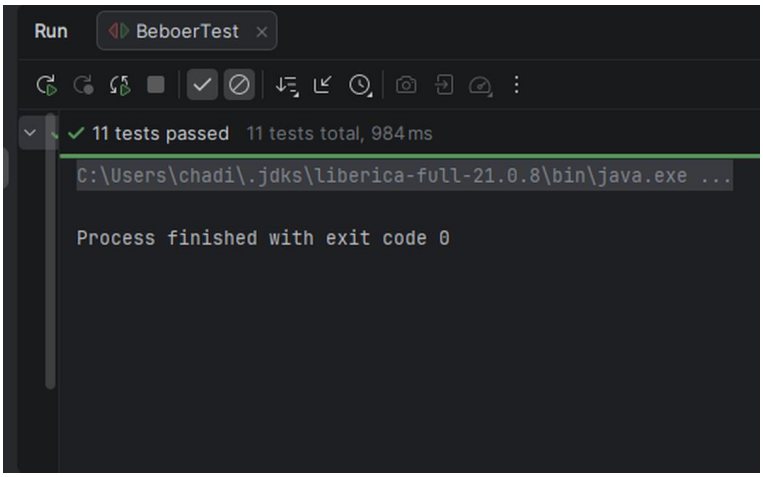
  assertThrows(IllegalArgumentException.class,
      () -> hanne.tilføjOpgave(bytteopgave));

  assertEquals(10, hanne.beregnPersonligePoint());
  assertTrue(hanne.getOpgaver().isEmpty());
}
```

Det er ikke nok, at der kommer en fejl. Hvis opgaven alligevel var blevet lagt på listen, eller saldoen var blevet ændret, ville data være forkerte, selvom brugeren fik en fejlbesked. Derfor tjekker de to sidste linjer, at intet er ændret.



Alle 11 tests bestod, som figur 9 viser.

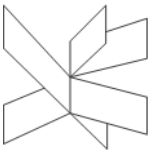


Figur 9: Resultat af BeboerTest i IntelliJ.

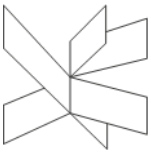
Black box-test af use cases

Use casene er testet gennem brugerfladen ud fra hovedscenariet og de vigtigste alternativer. Tabel 9 viser de 15 testcases. De dækker de use cases i administrationsprogrammet, der ændrer data, og gem og indlæs (FR12). Den eneste undtagelse er UC07 Slet opgave, som i stedet er testet med unittesten sletningAfBytteopgaveTilbagefoererPoint (tabel 8). Use cases, der kun viser data (UC04, UC08, UC09 og UC11), er testet indirekte, fordi vi kontrollerede resultatet af de andre tests i netop de skærmbilleder. Testene blev udført i rækkefølge på de samme data, så hver test bygger videre på den forrige.

ID	UC	Handling og input	Forventet resultat	Faktisk resultat
T01	UC01	Opret beboerne "Hanne" (hus 12, 01-05-1980) og "Jonas" (hus 14, 15-03-1975).	Begge vises i listen over beboere.	Bestået
T02	UC01	Opret beboer med tomt navn.	Fejlbesked "Navn skal udfyldes." Ingen beboer oprettes.	Bestået



ID	UC	Handling og input	Forventet resultat	Faktisk resultat
T03	UC13 + UC19	Opret fællesopgave "Havetjans" til 20 point. Afslut den og vælg Hanne.	Opgaven forsvinder fra listen. Hannes saldo stiger med 20.	Bestået
T04	UC14	Opret bytteopgave til 10 point med Hanne som bestiller (saldo 20).	Hannes saldo falder til 10.	Bestået
T05	UC14	Opret bytteopgave til 50 point med Hanne som bestiller (saldo 10).	Fejlbesked om for få point. Ingen opgave oprettes.	Bestået
T06	UC19	Afslut bytteopgaven fra T04.	Hanne vises ikke i dialogen. Jonas vælges og får 10 point.	Bestået
T07	UC12 + UC19	Opret grønt tiltag "Cyklet på arbejde i en uge" til 15 point og afslut det.	Fællespuljen stiger med 15. Ingen beboers saldo ændres.	Bestået
T08	UC19	Vælg Afslut opgave uden at vælge en opgave.	Fejlbesked "Vælg en opgave først."	Bestået
T09	UC10	Giv Hanne -100 point ved pointkorrektion (saldo 10).	Fejlbesked. Saldoen forbliver 10.	Bestået
T10	UC15	Nulstil Hannes point og bekræft.	Saldoen er 0, og nulstillingen vises i historikken.	Bestået
T11	UC02	Rediger Hannes husnummer fra 12 til 13 og gem.	Hannes husnummer ændres til 13 og vises korrekt i beboeroversigten.	Bestået
T12	UC03	Vælg Jonas, tryk "Slet" og bekræft sletningen.	Jonas fjernes fra beboerlisten.	Bestået
T13	UC06	Vælg en aktiv opgave, rediger beskrivelsen og gem.	Opgaven vises med den nye beskrivelse i opgaveoversigten.	Bestået
T14	UC16	Åbn Rapport efter de udførte tests.	Rapporten viser korrekt antal beboere, udførte opgaver, personlige point og fælles point.	Bestået



ID	UC	Handling og input	Forventet resultat	Faktisk resultat
T15	-	Luk programmet og start det igen.	Alle data fra T01–T14 er der stadig.	Bestået

Tabel 9: Black box-testcases.

Test af hjemmesiden

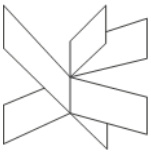
ID	UC/krav	Handling	Forventet resultat	Faktisk resultat
W01	UC17	Åbn index.html via IntelliJ.	Tallene på forsiden er de samme som i Rapport i programmet.	Bestået
W02	UC18	Afslut et grønt tiltag i programmet og genindlæs Grønne tiltag.	Det nye tiltag står øverst med dato og point.	Bestået
W03	NFR06	Åbn data.json og gennemse begge sider.	Ingen navne, husnumre eller individuelle saldi.	Bestået
W04	UC17	Åbn index.html direkte som fil.	Siden viser en besked om, at data ikke kunne hentes.	Bestået

Tabel 10: Test af hjemmesiden.

Sporbarhedsmatrix

Tabel 11 forbinder hvert krav med use case, kode og test.

Krav	UC	Klasse og metode	Test
FR01	UC01	BeboerController.opretBeboer, Beboer	T01, T02
FR02	UC02	BeboerController.redigerBeboer	T11
FR03	UC03	BeboerController.sletBeboer	T12
FR04	UC04	BeboerController.initialize	T01
FR05	UC05	OpgaveController.opretOpgave	T03, T04
FR06	UC06	OpgaveController.redigerOpgave	T13



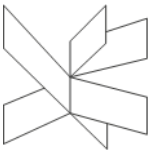
Krav	UC	Klasse og metode	Test
FR07	UC07	OpgaveController.sletOpgave, Beboer.fjernOpgave	sletningAfBytteopgaveTilbagefoererPoint (unittest)
FR08	UC08	OpgaveController.opdaterListe	T03
FR09	UC09	PointController, Beboer.beregnPersonligePoint	nyBeboerHarNulPoint
FR10	UC10	PointController.registrerPointaendring	T09
FR11	UC11	PointController.opdaterFaellesPoint	T07
FR12	–	FileDataManager.gem/hent, AppData	T15
FR13	UC17	index.html, kloeverly.js, WebDataManager	W01, W04
FR14	UC12	OpgaveController.afslutOpgave, FaellesPointpulje	T07
FR15	UC13	OpgaveController.afslutOpgave	T03
FR16	UC14	Beboer.tilføjOpgave, OpgaveController.afslutOpgave	T04–T06
FR17	UC18	groenne-tiltag.html, kloeverly.js, WebDataManager	W02, W03
FR18	UC15	PointController.nulstilPoint, Beboer.nulstilPersonligePoint	T10
FR19	UC16	RapportController.opdaterRapport	T14
FR20	UC19	OpgaveController.afslutOpgave	T03, T06–T08

Tabel 11: Sporbarhedsmatrix.

Test af ikke-funktionelle krav

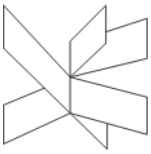
Tabel 12 viser, hvordan hvert ikke-funktionelt krav er testet. NFR03, NFR04 og NFR06 er dækket af testcases, som allerede er beskrevet. NFR01, NFR02 og NFR05 er testet særskilt.

ID	Krav	Handling	Forventet resultat	Faktisk resultat
N01	NFR01	Start på Dashboard, og klik på Beboere, Opgaver, Point og Rapport i menuen. Tæl klikkene.	1 klik til hvert skærmbillede.	Bestået



ID	Krav	Handling	Forventet resultat	Faktisk resultat
N02	NFR02	T02 og T05. Skriv desuden bogstaver i feltet Point og 0 point, når en opgave oprettes.	En fejlbesked, der forklarer fejlen, fx "Point skal være et tal.". Programmet lukker ikke.	Bestået
N03	NFR03	T03 og T13.	Ændringen vises i listen med det samme.	Bestået
N04	NFR04	T15: Luk programmet, og start det igen.	Alle data er der stadig.	Bestået
N05	NFR05	Slå netværket fra, start programmet, og opret en beboer.	Programmet starter uden login, og beboeren bliver oprettet.	Bestået
N06	NFR06	W03: Gennemse data.json og begge sider.	Ingen navne, husnumre eller individuelle saldi.	Bestået

Tabel 12: Test af ikke-funktionelle krav.

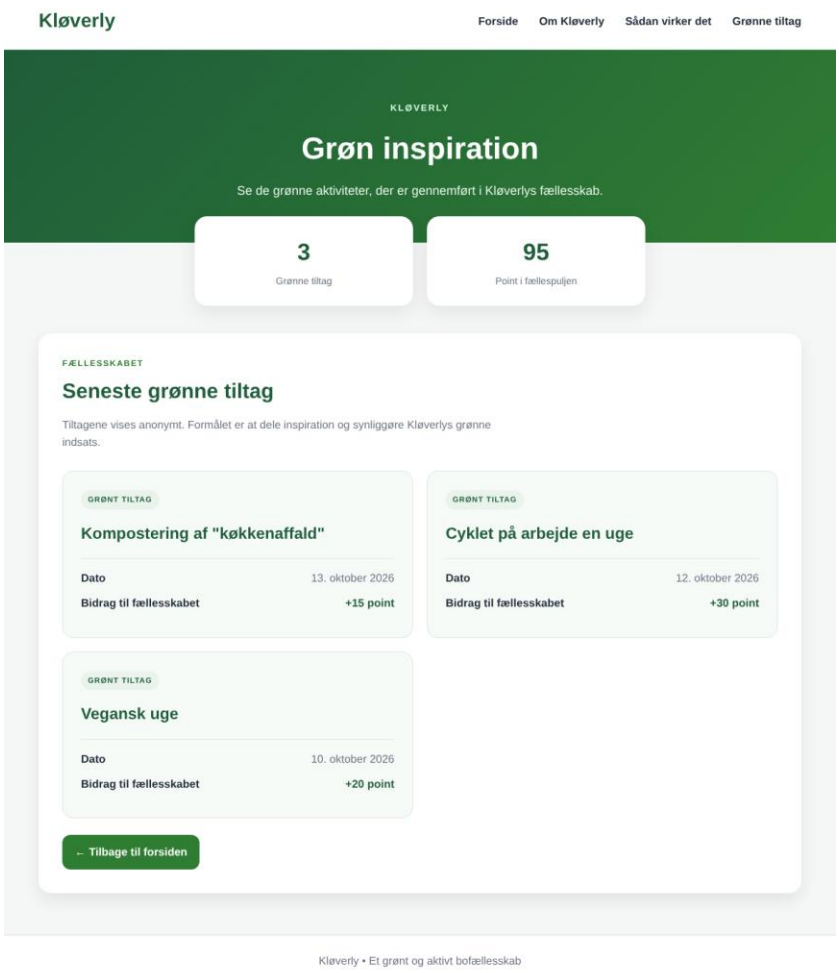


8. Resultater

Resultatet af projektet er et JavaFX-program til koordinatoren og en hjemmeside med to sider.

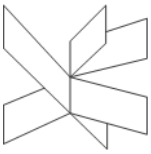
- 20 af 20 funktionelle krav er implementeret, og 20 er testet med et godkendt resultat.
- 6 af 6 ikke-funktionelle krav er testet med et godkendt resultat.
- Alle 11 unittests bestod, og 15 af 15 black box-testcases bestod.

Programmet skelner mellem de tre opgavetyper og de to slags point, som Bob beskriver. Personlige point kan ikke blive negative, og alle pointændringer gemmes som posteringer, så historikken bevares. Hjemmesiden viser landsbyens samlede tal og de grønne tiltag uden navne, og den opdateres, hver gang programmet gemmer.



Figur 10: Hjemmesidens side med grønne tiltag.

Testen viste ingen fejl på hjemmesiden. De grønne tiltag blev vist med korrekt dato og point, og hjemmesiden viste kun anonyme oplysninger som forventet.



9. Diskussion

I dette kapitel diskuterer vi løsningens svagheder, og hvad de betyder for, hvor godt problemet er løst.

Fortolkningen af bytteopgaver

Bob beskriver bytteopgaver som noget, "folk tilbyder", og han nævner "en pulje af bytteopgaver som man kan vælge fra". Vi har modelleret en bytteopgave som en bestilling: en beboer bestiller en ydelse og betaler, og en anden beboer udfører den. Det gør pointreglen enkel og sikrer, at pointene er reserveret. Men programmet kan ikke vise en liste af tilbud, som ingen endnu har bestilt. Hvis Bob mente et katalog af tilbud, opfylder løsningen kun en del af hans ønske.

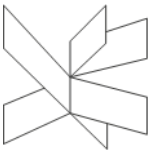
Vi vurderer, at denne fortolkning er en rimelig løsning i denne version af systemet, fordi den gør pointreglerne tydelige og nemme at implementere. Samtidig er vi opmærksomme på, at løsningen ikke fuldt ud understøtter idéen om en åben pulje af tilbud. Ved videreudvikling bør bytteopgaver derfor kunne oprettes som tilbud, som andre beboere efterfølgende kan vælge.

Pointlogik i controlleren

Reglen for at oprette en bytteopgave ligger i modellen, men reglerne for at afslutte en opgave ligger i OpgaveController. Det betyder, at reglerne ikke kan testes med JUnit uden brugerfladen, og at de skal skrives igen, hvis der laves en ny brugerflade, fx til skærmen. En bedre løsning ville være en metode som afslut(udfører) i Opgave-klasserne, så hver opgavetype selv kender sin pointregel.

Data og opgavernes tilstand

- Opgaver har ingen status. En afsluttet opgave fjernes fra listen over aktive opgaver og findes kun som reference i en pointpostering. Programmet kan derfor ikke vise en liste over afsluttede fællesopgaver.
- Bytteopgaver ligger hos bestilleren, mens fællesopgaver og grønne tiltag ligger i en fælles liste. To forskellige steder til samme slags data gør koden mere kompliceret.



- AppData er en statisk klasse, som alle controllere bruger direkte. Det er enkelt i et lille program, men gør det svært at teste controllerne.
- Når en bytteopgave slettes, fjernes bestillerens negative postering i stedet for at blive udlignet med en modpostering. Det giver den rigtige saldo, men historikken viser ikke, at opgaven har eksisteret. Det er i modstrid med princippet om, at historikken bevares, og en modpostering ville være mere konsekvent.
- Datafilen er binær. Hvis klassernes struktur ændres, kan en gammel fil blive inkompatibel og ikke længere kunne indlæses. Det kan medføre tab af adgang til de gemte data. Et tekstformat som JSON ville være mere robust og lettere at vedligeholde.
- Hvis en gemning fejler, fx fordi filen er låst, bliver fejlen kun skrevet i konsollen. Koordinatoren får ingen besked og tror, at data er gemt. Programmet burde vise en fejlbesked her, ligesom det gør ved ugyldige indtastninger.
- WebDataManager skriver data.json til stien src/main/resources/web, regnet fra den mappe, programmet startes i. Det virker, når vi starter programmet fra IntelliJ, men ikke hvis programmet pakkes og installeres på Bobs computer. Stien burde kunne indstilles.

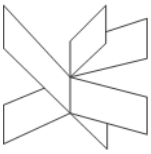
Hvad mangler i forhold til Bobs ønsker

Skærmvisningen, målene for fællespuljen og pointboostet er ikke implementeret, se afgrænsningen.

Det er samtidig ikke afklaret, hvilke oplysninger skærmen bør vise. Hvis beboernes navne og personlige point vises offentligt i fælleshuset, kan det skabe en form for konkurrence, som Bob netop ønsker at undgå. En videreudvikling bør derfor tage stilling til, om skærmen i stedet skal vise anonyme eller samlede oplysninger.

Persondata

Programmet gemmer beboernes fødselsdato (FR01), men fødselsdatoen indgår ikke i nogen af programmets regler eller beregninger, og Bob nævner den ikke i interviewet. Efter princippet om dataminimering i databeskyttelsesforordningen må man kun



indsamle de oplysninger, der er nødvendige til formålet (Europa-Parlamentet og Rådet, 2016). Fødselsdatoen kunne bruges til at skelne mellem to beboere med samme navn, men det klarer husnummeret i de fleste tilfælde. Vi burde derfor have spurgt Bob, om han har brug for den, og ellers fjernet feltet.

Test

Unittestene dækker kun Beboer, og black box-testene er udført af os selv. Løsningen er ikke afprøvet af en rigtig bruger. Vi ved derfor ikke, om brugerfladen er let at bruge for Bob.

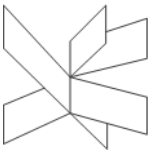
Der er heller ingen automatiske tests af controllerne. Reglerne for at afslutte en opgave er derfor kun testet manuelt gennem brugerfladen (T03, T06 og T07). Testene af hjemmesiden er også manuelle. Hvis koden bliver ændret senere, skal de tests gentages i hånden, og det er let at glemme. Det er endnu en grund til at flytte reglerne for afslutning ind i modelklasserne, så de kan dækkes af unittests.

Testene viste, at alle 20 funktionelle krav og alle 6 ikke-funktionelle krav blev opfyldt i de gennemførte tests. Alle 11 unittests og 15 black box-tests bestod. Der er derfor ikke identificeret krav, som fejlede i testen. En begrænsning er dog, at systemet ikke er brugertestet af Bob eller andre eksterne brugere, så vi kan ikke konkludere, hvor intuitiv brugerfladen er i praksis.

Brug af generativ AI

Vi har brugt generativ AI meget i projektet, som beskrevet i kapitel 3. Uden det havde vi ikke nået en fungerende løsning på én uge. Men vi oplevede også flere gange, at det, AI lavede, var forkert eller ikke passede med resten. Tre eksempler:

- Unittestene var skrevet efter en tidligere pointregel. Da vi ændrede reglen, så point først gives ved afslutning, fejlede fem af de ti tests, og de måtte skrives om.
- I de første sekvensdiagrammer hed livslinjerne noget andet end klasserne i koden, og fragmenterne sad forkert.
- Klassediagrammet havde forkerte returtyper, fx getOpgaver() : void.

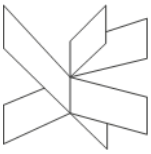


Vi fandt fejlene, fordi vi sammenlignede diagrammer, kode og rapport og kørte testene. Her fik vi brug for sporbarheden mellem krav, design, kode og test i praksis, for det var den, der fangede fejlene. Den største risiko ved AI er, at noget kan se færdigt ud, før man har forstået det. Derfor har vi gennemgået koden og diagrammerne sammen, så vi begge kan forklare løsningen.

Vandfaldsmodellen på en uge

Vandfaldsmodellen forudsætter, at kravene er stabile, før designet begynder. I praksis ændrede vi kravene undervejs, fx da vi besluttede, at point først gives ved afslutning, og tilføjede FR20. Use case-beskrivelserne blev også skrevet efter koden. Det betyder, at dokumentationen er rettet efter koden og ikke omvendt. Planen for ugen står i projektbeskrivelsen i bilag 7.

Vi vurderer, at vandfaldsmodellen var nyttig, fordi den gav en tydelig rækkefølge mellem krav, analyse, design, implementering og test. Samtidig passede den ikke perfekt til vores projekt, fordi flere krav og designvalg ændrede sig under implementeringen. En mere iterativ model kunne have gjort det lettere at afprøve funktionerne tidligere og løbende rette krav og diagrammer, så vi havde undgået en del efterfølgende rettelser. Til et kort projekt som dette gav vandfaldsmodellen dog en enkel struktur og gjorde sporbarheden mellem krav, design, kode og test tydelig.



10. Konklusion og anbefalinger

Vi har udviklet et IT-system, der hjælper koordinatoren i Kløverly med at registrere de tre typer opgaver og de to typer point.

Hvilke krav kan udledes af interviewet, og hvordan kan de testes? Vi har udledt 20 nummererede funktionelle krav af interviewet med Bob, og hvert krav har et citat fra interviewet som grundlag. De 6 ikke-funktionelle krav er formuleret, så de kan måles i en test.

Hvordan kan pointreglerne modelleres? Pointene er modelleret som pointposteringer. Saldoen beregnes ud fra historikken, og den kan ikke blive negativ. En bytteopgave reserverer point, når den oprettes, mens de andre opgavetyper først giver point, når de afsluttes.

Hvordan kan data vises på hjemmesiden uden personoplysninger? Programmet skriver en datafil med kun samlede tal og grønne tiltag, og JavaScript henter filen og viser indholdet. Navnene bliver aldrig skrevet til filen, så hjemmesiden kan ikke vise dem.

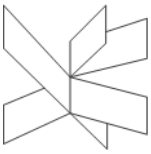
Hvordan kan vi dokumentere, at løsningen opfylder kravene? Med 11 unittests, 15 black box-tests, 4 tests af hjemmesiden og en test af hvert ikke-funktionelt krav. Sporbarhedsmatricen forbinder hvert krav med use case, kode og test.

Testresultaterne viser, at alle 20 funktionelle krav og alle 6 ikke-funktionelle krav blev opfyldt. Desuden bestod alle 11 unittests, alle 15 black box-tests og alle 4 tests af hjemmesiden.

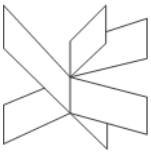
Løsningen dækker ikke alle Bobs ønsker. Skærmvisningen, målene for fællespuljen og pointboostet er ikke implementeret.

Anbefalinger

- Afklar med Bob, hvad skærmen i fælleshuset skal vise, før den udvikles.
- Flyt reglerne for at afslutte en opgave fra controlleren til modelklasserne, og giv opgaverne en status.



- Tilføj mål for fællespuljen, så hjemmesiden kan vise, hvor tæt landsbyen er på næste fælles aktivitet.
- Afklar med Bob, om bytteopgaver skal kunne oprettes som tilbud uden bestiller.
- Afklar med Bob, om fødselsdatoen er nødvendig, og fjern feltet, hvis den ikke er.
- Gem data i et tekstformat som JSON, så data bliver lettere at læse, vedligeholde og migrere, hvis klassernes struktur ændres.



11. Referencer

Anthropic. (2026). Claude [Stor sprogmodel]. https://claude.ai/

Europa-Parlamentet og Rådet. (2016). Forordning (EU) 2016/679 om beskyttelse af fysiske personer i forbindelse med behandling af personoplysninger (databeskyttelsesforordningen). https://eur-lex.europa.eu/eli/reg/2016/679/oj

Mozilla. (u.å.). Using the Fetch API. MDN Web Docs. Hentet 14. oktober 2026 fra https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

Nielsen, J. (1994). 10 usability heuristics for user interface design. Nielsen Norman Group. Hentet 15. oktober 2026 fra https://www.nngroup.com/articles/ten-usability-heuristics/

OpenAI. (2026). ChatGPT [Stor sprogmodel]. https://chatgpt.com/

OpenJFX. (u.å.). JavaFX documentation. Hentet 14. oktober 2026 fra https://openjfx.io/

Oracle. (2023). Java Platform, Standard Edition 21 API specification. https://docs.oracle.com/en/java/javase/21/docs/api/

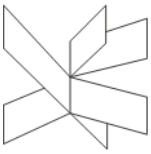
Royce, W. W. (1970). Managing the development of large software systems. I Proceedings of IEEE WESCON (s. 1–9). IEEE.

The JUnit Team. (u.å.). JUnit 5 user guide. Hentet 14. oktober 2026 fra https://junit.org/junit5/docs/current/user-guide/

VIA University College. (2024). Guide til projektarbejde – Projektrapport. VIA Ingeniøruddannelser.

VIA University College. (2025a). Kløverly – SW-SEP1-case, efterår 2025.

VIA University College. (2025b). Kursusbeskrivelse for TS-SEP1.



12. Bilag

- Bilag 1: Brainstorm og aktører
- Bilag 2: Kravspecifikation
- Bilag 3: Use case-beskrivelser UC01–UC19
- Bilag 4: UML-diagrammer (use case-diagram, domænemodel, klassediagram, 19 aktivitetsdiagrammer og 3 sekvensdiagrammer)
- Bilag 5: Gruppekontrakt
- Bilag 6: Brugermanual
- Bilag 7: Projektbeskrivelse

Kildekoden afleveres separat og findes også på GitHub:

https://github.com/chadiElKhatib/Sep1_Mehmet_Chadi_Reeksamen